```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

```
df=pd.read_csv('/content/supply_chain.csv')
df.head(2)
```

	Product type	SKU	Price	Availability	Number of products sold	Revenue generated	Customer demographics	Stock levels	Lead times	Order quantities	...	Location	Lea tim
0	haircare	SKU0	69.808006	55	802	8661.996792	Non-binary	58	7	96	...	Mumbai	2
1	skincare	SKU1	14.843523	95	736	7460.900065	Female	53	30	37	...	Mumbai	2

2 rows × 24 columns

```
!pip install plotly
```

```
Requirement already satisfied: plotly in /usr/local/lib/python3.13/dist-packages (5.24.1)
Requirement already satisfied: tenacity>=6.2.0 in /usr/local/lib/python3.13/dist-packages (from plotly) (9.1.4)
Requirement already satisfied: packaging in /usr/local/lib/python3.13/dist-packages (from plotly) (26.3)
```

```
import plotly.express as px
import plotly.io as pio
import plotly.graph_objects as go
pio.templates.default='plotly_white'
```

```
df.describe()
```

	Price	Availability	Number of products sold	Revenue generated	Stock levels	Lead times	Order quantities	Shipping times	Shipping costs	Lead time
count	100.000000	100.000000	100.000000	100.000000	100.000000	100.000000	100.000000	100.000000	100.000000	100.000000
mean	49.462461	48.400000	460.990000	5776.048187	47.770000	15.960000	49.220000	5.750000	5.548149	17.080000
std	31.168193	30.743317	303.780074	2732.841744	31.369372	8.785801	26.784429	2.724283	2.651376	8.846251
min	1.699976	1.000000	8.000000	1061.618523	0.000000	1.000000	1.000000	1.000000	1.013487	1.000000
25%	19.597823	22.750000	184.250000	2812.847151	16.750000	8.000000	26.000000	3.750000	3.540248	10.000000
50%	51.239831	43.500000	392.500000	6006.352023	47.500000	17.000000	52.000000	6.000000	5.320534	18.000000
75%	77.198228	75.000000	704.250000	8253.976921	73.000000	24.000000	71.250000	8.000000	7.601695	25.000000
max	99.171329	100.000000	996.000000	9866.465458	100.000000	30.000000	96.000000	10.000000	9.929816	30.000000

```
Start coding or generate with AI.
```

now let's get started with analysis the supply chain by lokinng at the relationship b/w the price of the products and the revenue genreted by them:

```
#!pip install statsmodels
```

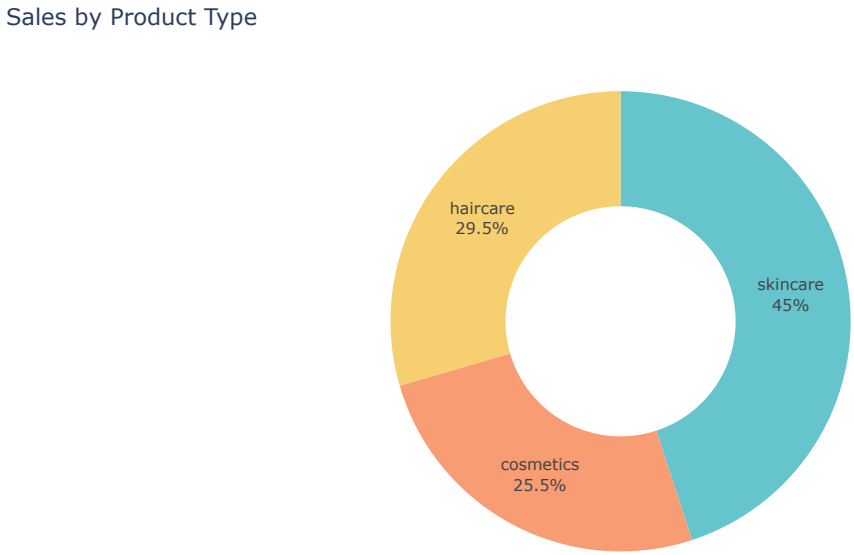
```
fig=px.scatter(df,x='Price',
               y='Revenue generated',
               color='Product type',
               hover_data=['Number of products sold'],
               trendline='ols')
fig.show()
```



thus,the company derives more revenue from skincare products,and the higher the price of skincare products,the more revenue they genrete.now let's have a look at the sales by product type: **bold text**

```
sales_data=df.groupby('Product type')['Number of products sold'].sum().reset_index()
#sales_data

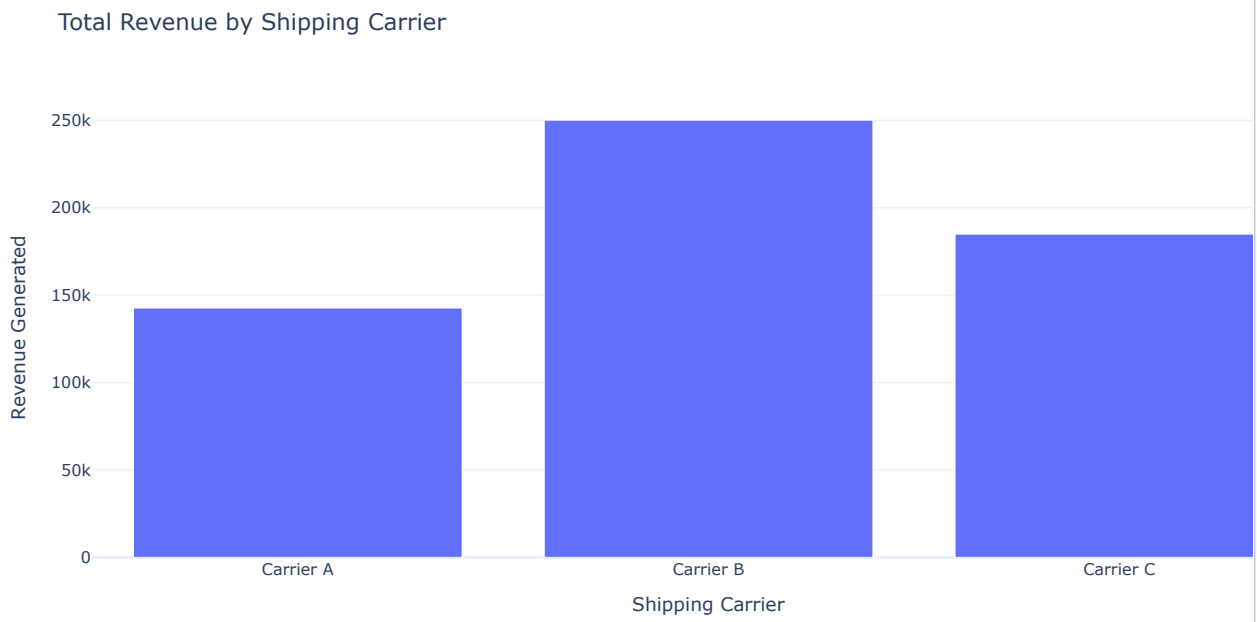
pie_chart=px.pie(sales_data,names='Product type',
                 values='Number of products sold',
                 title='Sales by Product Type',
                 hover_data=['Number of products sold'],
                 hole=0.5,
                 color_discrete_sequence=px.colors.qualitative.Pastel)
pie_chart.update_traces(textposition='inside',textinfo='percent+label')
pie_chart.show()
```



Start coding or generate with AI.

so 45% of the bussiness comes from skincare products ,29.5% from hairecare,25.5% from cosemetic .now let's have a look at the total revenue genreted from shipping carriers:bold text

```
total_revenue=df.groupby('Shipping carriers')['Revenue generated'].sum().reset_index()
fig=go.Figure()
fig.add_trace(go.Bar(x=total_revenue['Shipping carriers '],
                     y=total_revenue['Revenue generated']))
fig.update_layout(title='Total Revenue by Shipping Carrier',
                  xaxis_title='Shipping Carrier',
                  yaxis_title='Revenue Generated')
fig.show()
```



so the company is using three cariers for tranportation ,and carrier b helps the company in gnreting more revenue .now let's have a look at the average lead time and average manufacturing costs for all products of the company:

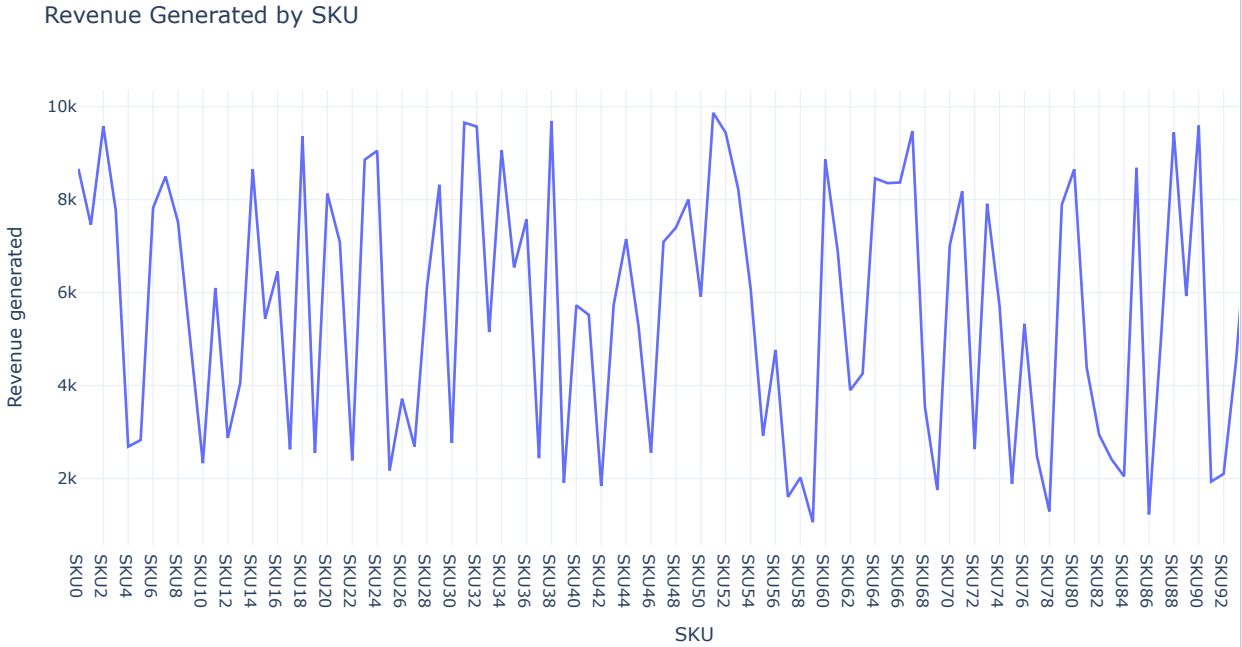
```
avg_lead_time=df.groupby('Product type')['Lead time'].mean().reset_index()
#avg_lead_time
avg_manufacturing_cost=df.groupby('Product type')['Manufacturing costs'].mean().reset_index()
result=pd.merge(avg_lead_time,avg_manufacturing_cost,on='Product type')
result.rename(columns={'Lead time':'Average Lead Time',
                       'Manufacturing costs':'Average Manufacturing Cost'},inplace=True)
print(result)
```

```
  Product type  Average Lead Time  Average Manufacturing Cost
0    cosmetics          13.538462                   43.052740
1     haircare          18.705882                   48.457993
2     skincare          18.000000                   48.993157
```

```
Start coding or generate with AI.
```

ANALYZING SKU'S (STOCK KEEPING UNITS) THERE'S IS A COLUMNIN THE DATASET AS A SKU'S .YOU MUST HAVE HEARD IT FOR THE VERY FIRST TIME .SO,SKU STANDS FOR STOCK KEEPING UNITS.THEY'RE LIKE SPECIAL CODES THAT HELP COMPANIES KEEP TRACK OF THE ALL DIFFRENT THINGS THEY HAVE FOR SALE.IMAGINE YOU HAVE A LARGE TOY STORE WITH LOTS OF TOY .EACH TOY IS DIFFRENT AND HAS ITS NAME AND PRICE BUT WHEN YOU WANT TO KNOW HOW MMANY YOU HAVE LEFT ,YOU NEED A WAT TO IDENTIFY THEM. SO YOU GIVE EACH TOY A UNIQUE CODE.LIKE SECRET NUMBER ONLY THE STORE KNOWS. THE SECRET NUMBER IS CALLED SKU I HOPE YOU HAVE NOW UNDERSTND WHAT'S SKU;S NOW ANALYSIS THE REVENUE GENRETED BY EACH SKU

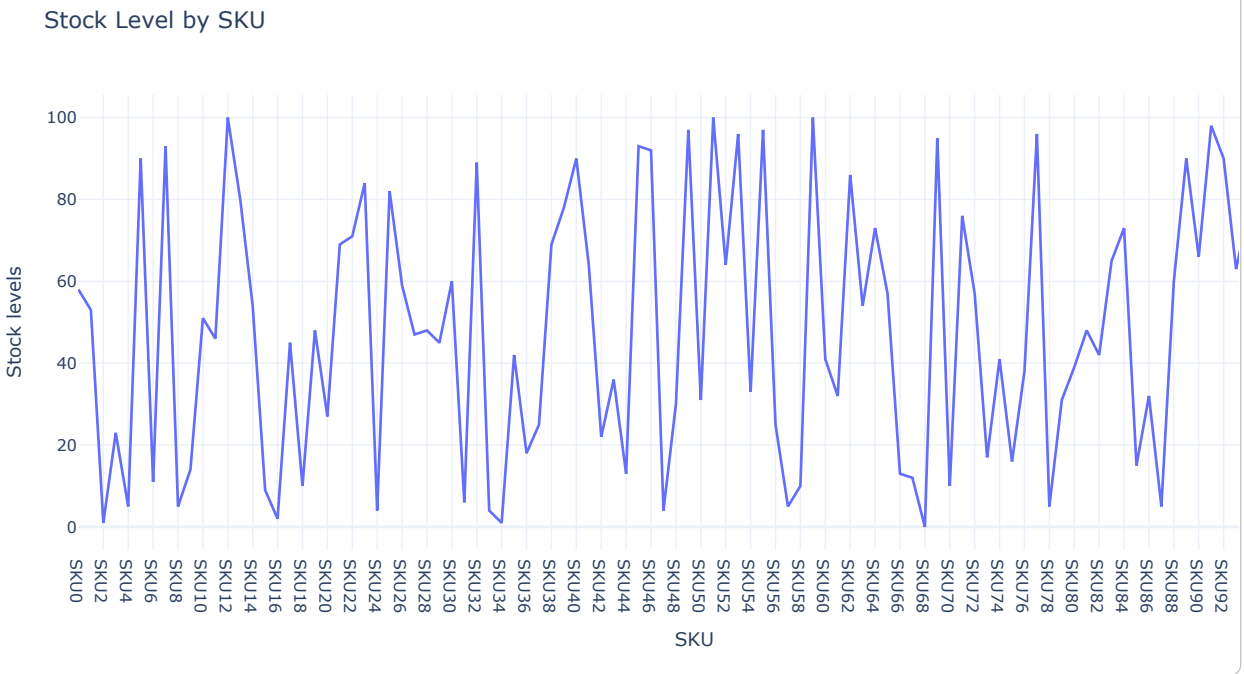
```
revenue_chart=px.line(df,x='SKU',
                      y='Revenue generated',
                      title='Revenue Generated by SKU')
revenue_chart.show()
```




```
Start coding or generate with AI.
```

ther's another column in the dataset as stock levels .stock levels refer to the number of products a store or business has in its inventory.now let's have a look at the stock levels of each SKU:**bold text**

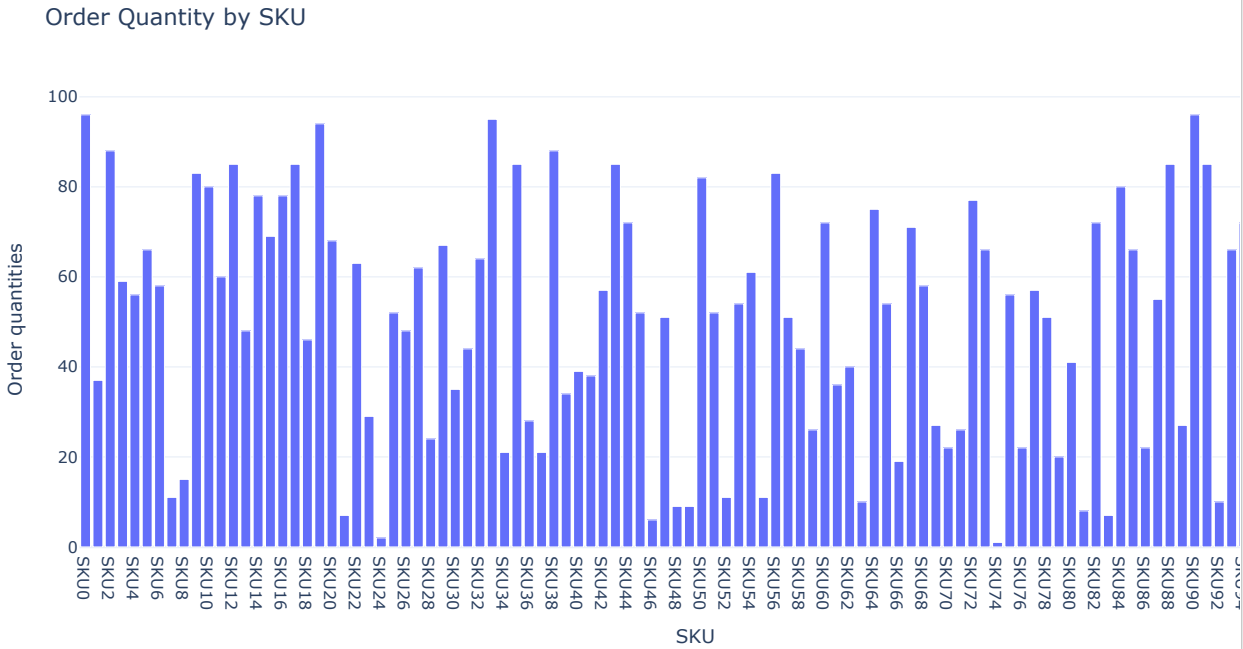
```
stock_chart=px.line(df,x='SKU',
                    y='Stock levels',
                    title='Stock Level by SKU')
stock_chart.show()
```



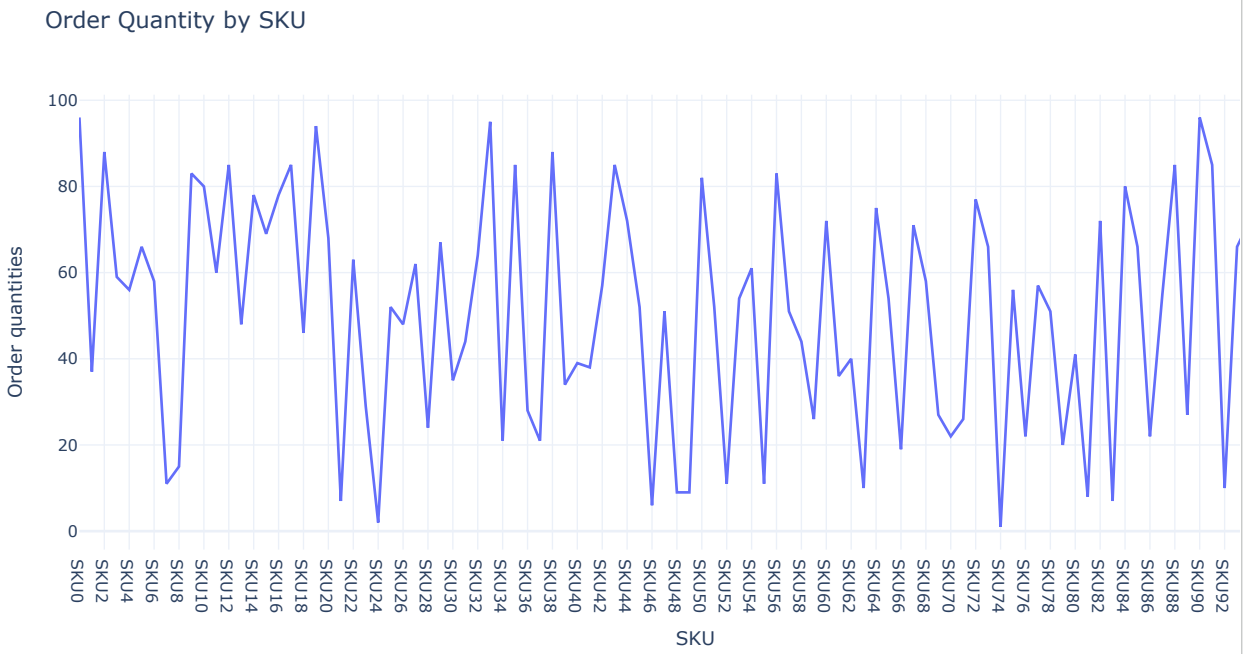

```
Start coding or generate with AI.
```

NOW LET'S HAVE A LOOK AT THE ORDER QUANTITY OF EACH SKU:

```
order_quantity_chart=px.bar(df,x='SKU',y='Order quantities',title='Order Quantity by SKU')
order_quantity_chart.show()
```

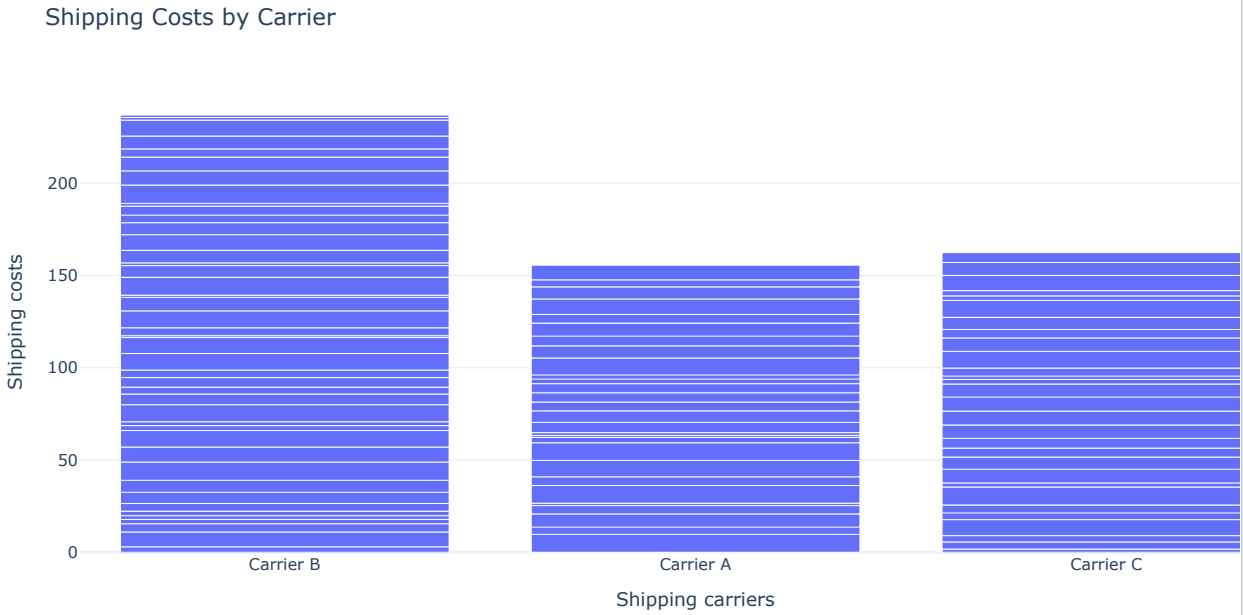



```
order_quantity_chart=px.line(df,x='SKU',y='Order quantities',title='Order Quantity by SKU')
order_quantity_chart.show()
```



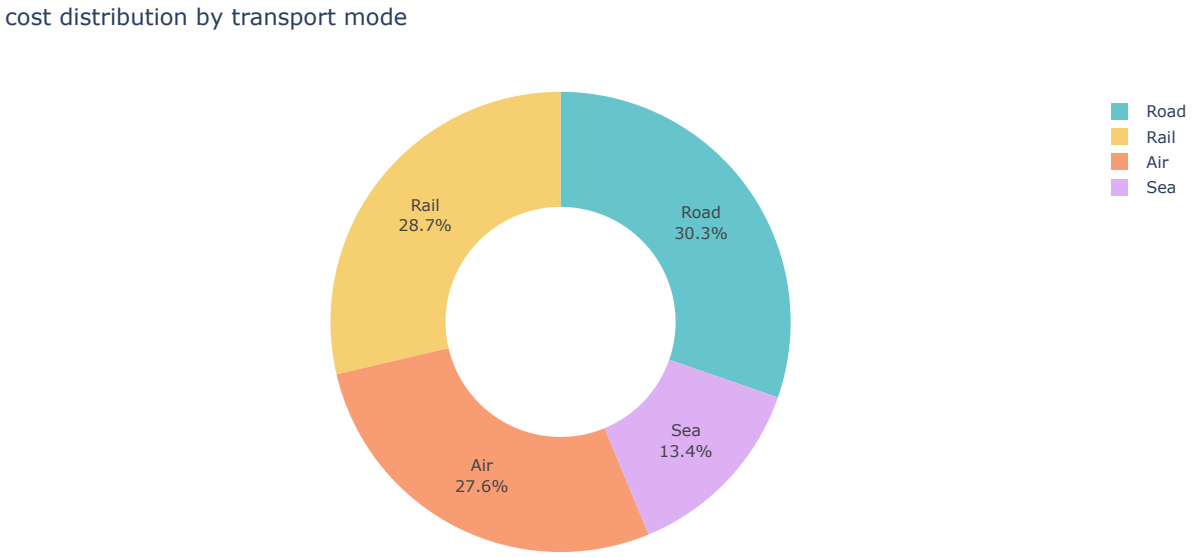
COST ANALYSIS now let's analyzing the shipping cost of carriers:

```
shipping_cost_chart=px.bar(df,x='Shipping carriers',y='Shipping costs',title='Shipping Costs by Carrier')
shipping_cost_chart.show()
```



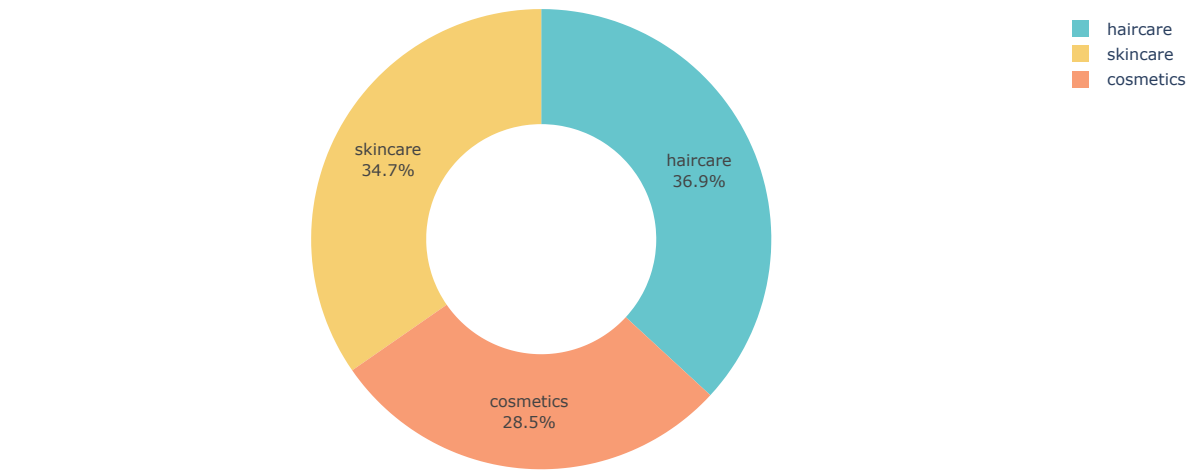

```
transport_data=df.groupby('Transportation modes')['Costs'].sum().reset_index()
#transport_data

pie_chart=px.pie(transport_data,names='Transportation modes',
                 values='Costs',
                 title='cost distribution by transport mode',
                 hover_data=['Costs'],
                 hole=0.5,
                 color_discrete_sequence=px.colors.qualitative.Pastel)
pie_chart.update_traces(textposition='inside',textinfo='percent+label')
pie_chart.show()
```




```
product_type_data=df.groupby('Product type')['Defect rates'].mean().reset_index()
#transport_data

pie_chart=px.pie(product_type_data,names='Product type',
                 values='Defect rates',
                 title='avg product type by defect rates',
                 hover_data=['Defect rates'],
                 hole=0.5,
                 color_discrete_sequence=px.colors.qualitative.Pastel)
pie_chart.update_traces(textposition='inside',textinfo='percent+label')
pie_chart.show()
```


avg product type by defect rates
haircare
skincare
cosmetics
skincare
34.7%
haircare
36.9%
cosmetics
28.5%